

Django App Blueprint: Advise Expert Sessions

https://gemini.google.com/app/9bc365232e31ef3d

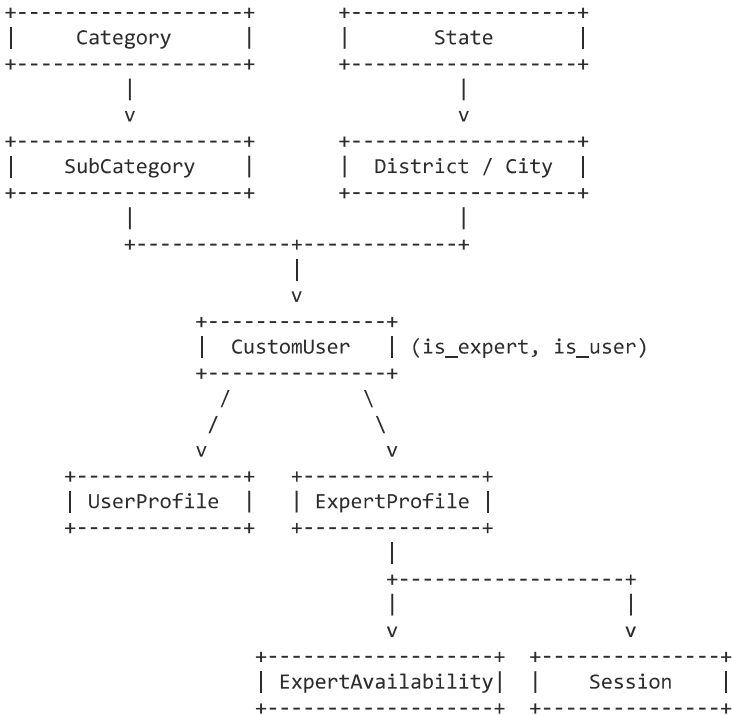
User prompt: i am doing internship , i have a team of three people and we have to create our first app using django the description of the app is as follows the name of the app is advise the concept of this app is a user can request a scheduled session with an expert in that field it will works as 1 . first the home page where we can see a navbar in this home page, how to use , about, contact, login/register these are options on the navbar now at login/register you have two options register as expert and register as user now if you want to login as user you are asked some basic info that is necessary like name, email , phone number, password and address and address in the format of state, district , city which is saved in our database and can be shown in the form in drop down list format and now as when you register as an expert you it happens in three steps first is basic info and password and at next step you should asked to select the category , then sub-category which is also stored in our database and the expert can select from it and qualification then experience , location and description about yourself and at next part he will be showed the final profile of him and after clicking complete registration the profile will be created now that was about the login and register part 2. now come to the logic part that how the app will works now on after login as expert you can schedule calendar like when you are available and what is the time slot and what is the price per hour 3. now when you login as user you can see the dashboard then you can search the expert by name , category , subcategory also you can filter the results by the location ,price, time slot. 4. this was about the working now additional points are when a user wants to book a expert the expert will get a request of the user through notification now when expert accept the request new session is generated as the user will get another notification to make payment and confirm the booking this will only available for 24hr to make the payment otherwise the request is rejected and when a expert is booked then when another user tries to request for same slot he will showed as the expert is unavailable for this slot also the expert have 3 options to handle the request as accept, reject, and post-pond now this is the overall concept of our project now guide me to develop the complete working and exception handled project

Response: This is a solid, practical full-stack Django project for a team of three. To build **Advise** cleanly without getting overwhelmed, you should break the project into **modular components**, establish a clear **Database Schema**, and divide responsibilities efficiently among your team.

Here is your complete blueprint to build a working, production-ready app.

1. Project Architecture & Database Design

Your app relies on two main user roles (**User** and **Expert**) extending Django's custom user model, plus relational models for location, categorization, availability, and session bookings.



Models Setup (models.py)

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from django.utils import timezone
from datetime import timedelta

class UserRole(models.TextChoices):
    USER = 'USER', 'User'
    EXPERT = 'EXPERT', 'Expert'

class CustomUser(AbstractUser):
    role = models.CharField(max_length=10, choices=UserRole.choices, default=UserRole.USER)
    phone_number = models.CharField(max_length=15, unique=True)

class State(models.Model):
    name = models.CharField(max_length=100)

class District(models.Model):
    state = models.ForeignKey(State, on_delete=models.CASCADE, related_name='districts')
    name = models.CharField(max_length=100)

class City(models.Model):
    district = models.ForeignKey(District, on_delete=models.CASCADE, related_name='cities')
    name = models.CharField(max_length=100)

class UserProfile(models.Model):
    user = models.OneToOneField(CustomUser, on_delete=models.CASCADE, related_name='user_profile')
    state = models.ForeignKey(State, on_delete=models.SET_NULL, null=True)
    district = models.ForeignKey(District, on_delete=models.SET_NULL, null=True)
    city = models.ForeignKey(City, on_delete=models.SET_NULL, null=True)

class Category(models.Model):
    name = models.CharField(max_length=100)
```

```
class SubCategory(models.Model):
    category = models.ForeignKey(Category, on_delete=models.CASCADE, related_name='subcategories')
    name = models.CharField(max_length=100)

class ExpertProfile(models.Model):
    user = models.OneToOneField(CustomUser, on_delete=models.CASCADE, related_name='expert_profile')
    category = models.ForeignKey(Category, on_delete=models.SET_NULL, null=True)
    subcategory = models.ForeignKey(SubCategory, on_delete=models.SET_NULL, null=True)
    qualification = models.CharField(max_length=255)
    experience_years = models.PositiveIntegerField()
    state = models.ForeignKey(State, on_delete=models.SET_NULL, null=True)
    district = models.ForeignKey(District, on_delete=models.SET_NULL, null=True)
    city = models.ForeignKey(City, on_delete=models.SET_NULL, null=True)
    hourly_rate = models.DecimalField(max_digits=8, decimal_places=2, default=0.00)
    bio = models.TextField()

class ExpertAvailability(models.Model):
    expert = models.ForeignKey(ExpertProfile, on_delete=models.CASCADE, related_name='availabilities')
    date = models.DateField()
    start_time = models.TimeField()
    end_time = models.TimeField()
    is_booked = models.BooleanField(default=False)

class SessionStatus(models.TextChoices):
    REQUESTED = 'REQUESTED', 'Requested'
    ACCEPTED = 'ACCEPTED', 'Accepted (Payment Pending)'
    POSTPONED = 'POSTPONED', 'Postponed'
    CONFIRMED = 'CONFIRMED', 'Confirmed (Paid)'
    REJECTED = 'REJECTED', 'Rejected'
    EXPIRED = 'EXPIRED', 'Expired'

class SessionBooking(models.Model):
    user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='bookings')
    expert = models.ForeignKey(ExpertProfile, on_delete=models.CASCADE, related_name='sessions')
    slot = models.ForeignKey(ExpertAvailability, on_delete=models.CASCADE)
    status = models.CharField(max_length=20, choices=SessionStatus.choices, default=SessionStatus.REQUESTED)
    created_at = models.DateTimeField(auto_now_add=True)
    payment_deadline = models.DateTimeField(null=True, blank=True)

    def is_payment_expired(self):
        return self.status == SessionStatus.ACCEPTED and timezone.now() > self.payment_deadline
```

2. Step-by-Step Feature Implementation

Feature 1: Dynamic Location Dropdowns

Use **AJAX (JavaScript)** so selecting a **State** dynamically fetches its **Districts**, and selecting a District dynamically populates **Cities**.

```
// Example AJAX fetching districts based on state selection
document.getElementById('id_state').addEventListener('change', function() {
    let stateId = this.value;
    fetch(`/api/get-districts/?state_id=${stateId}`)
        .then(response => response.json())
        .then(data => {
            let districtSelect = document.getElementById('id_district');
            districtSelect.innerHTML = '<option value="">Select District</option>';
            data.forEach(item => {
                districtSelect.innerHTML += `<option value="${item.id}">${item.name}</option>`;
            });
        });
});
```

Feature 2: Multi-step Expert Registration

Instead of building 3 separate backend views for registration, use standard Django Session Storage to carry multi-step data across pages before saving:

- **Step 1:** Validate basic info (name, email, password) → store in request.session['expert_step_1'].
- **Step 2:** Form collects category, subcategory, experience, bio → store in request.session['expert_step_2'].
- **Step 3:** Profile summary page with a single **"Complete Registration"** button. Upon submit, create both CustomUser and ExpertProfile inside an atomic transaction:

```
from django.db import transaction

@transaction.atomic
def complete_expert_registration(request):
    data1 = request.session.get('expert_step_1')
    data2 = request.session.get('expert_step_2')

    user = CustomUser.objects.create_user(
        username=data1['email'],
        email=data1['email'],
        password=data1['password'],
        role=UserRole.EXPERT,
        phone_number=data1['phone']
    )
    ExpertProfile.objects.create(
        user=user,
        category_id=data2['category_id'],
        subcategory_id=data2['subcategory_id'],
        qualification=data2['qualification'],
        experience_years=data2['experience'],
        hourly_rate=data2['hourly_rate'],
```

```
        bio=data2['bio']
    )
    # Clear session data
    del request.session['expert_step_1']
    del request.session['expert_step_2']
```

Feature 3: Dynamic Search & Filtering

Use Django ORM queries (Q objects) to handle multi-parameter filter combinations easily:

```
from django.db.models import Q

def search_experts(request):
    query = request.GET.get('q', '')
    category_id = request.GET.get('category')
    max_price = request.GET.get('max_price')
    city_id = request.GET.get('city')

    experts = ExpertProfile.objects.all()

    if query:
        experts = experts.filter(
            Q(user__first_name__icontains=query) |
            Q(user__last_state__icontains=query) |
            Q(bio__icontains=query)
        )
    if category_id:
        experts = experts.filter(category_id=category_id)
    if max_price:
        experts = experts.filter(hourly_rate__lte=max_price)
    if city_id:
        experts = experts.filter(city_id=city_id)

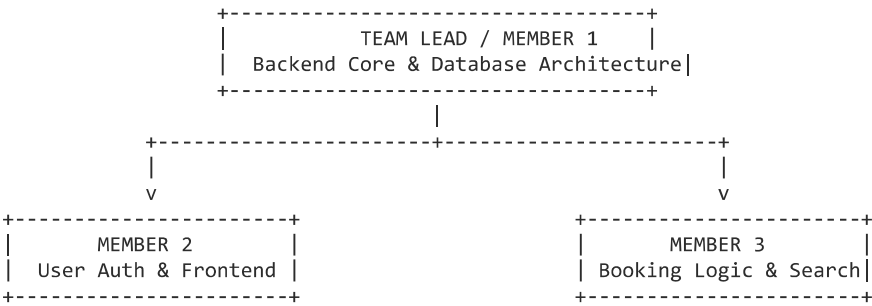
    return render(request, 'dashboard/search.html', {'experts': experts})
```

3. Crucial Logic & Exception Handling

Challenge / Edge Case	Cause	Solution
Double Booking / Race Condition	Two users request the exact same available slot simultaneously.	Use Django's <code>select_for_update()</code> inside an <code>@transaction.atomic</code> block when requesting or accepting a slot.
24-Hour Payment Expiry	User does not pay within 24 hours of acceptance.	1. Calculate <code>payment_deadline = timezone.now() + timedelta(hours=24)</code> when accepted. 2. Run a periodic background task (via Celery or a custom Django Management Command) that marks unpaid bookings as EXPIRED and re-opens the slot (<code>is_booked = False</code>).
Postpone Logic	Expert postpones the session instead of accepting or rejecting.	Store a proposed datetime in <code>SessionBooking</code> . Send a notification to the user asking to Accept Postponed Time or Cancel Request .
Inactive/Incomplete Multi-step Session	User leaves mid-way during expert registration.	Set a timeout on session state or clear registration session keys upon visiting the home page/login page.

4. Work Division for 3 Team Members

To work effectively without merge conflicts, divide the project by functional modules:



- **Member 1 (Backend Core & Models Lead):**
 - Database schema design, Custom User models, and Location models (State, District, City).
 - Category and Subcategory DB seeds/fixtures.
 - Expert Availability schedule creation and management views.
- **Member 2 (Frontend & Auth Specialist):**
 - UI layout (HTML/CSS/Bootstrap or Tailwind), Navbar, Landing page.
 - Dynamic AJAX dependent dropdowns (State → District → City).
 - Multi-step Expert Registration wizard UX and user registration/login forms.
- **Member 3 (Search & Booking Logic Lead):**
 - Expert filtering system (Name, Price, Category, Time Slot, Location).
 - Session booking flow (Request, Accept, Reject, Postpone actions).

- Expiration logic (24-hour timer check) and notification system (Internal DB notifications/Email).

User prompt: give me a roadmap to complete this project also tell how the tables are created and related in mysql and also tell how to create this tables

Response:

User prompt: give me a roadmap to complete this project also tell how the tables are created and related in mysql and also tell how to create this tables and also give me complete file structure diagram like how it will look after the complete project

Response: Here is your complete blueprint: a step-by-step development roadmap, the precise MySQL database relational structure with SQL creation scripts, and the final Django project file layout.

1. Step-by-Step Development Roadmap

To finish this project on time without overlapping tasks, follow this 4-phase milestone path:

[Phase 1: Setup & Data]	→	[Phase 2: Authentication]	→	[Phase 3: Scheduling & Search]	→	[Phase 4: Booking Flow]
- Base Django Setup		- Custom User Model		- Expert Slot Management		- Notification System
- MySQL Connection		- Cascading Dropdowns		- Dynamic Filtering		- 24-Hour Expiry Logic
- Master Location Data		- 3-Step Expert Signup		- Expert Profile Page		- Rejection / Postpone Flow

Phase 1: Foundation & Location Architecture

- **Task 1.1:** Initialize the Django project, connect it to MySQL, and define the custom user model.
- **Task 1.2:** Create models for State, District, City, Category, and SubCategory.
- **Task 1.3:** Seed master data into your MySQL database (or write a Django custom command to seed Indian states, districts, and categories).
- **Task 1.4:** Build AJAX API endpoints for dependent cascading dropdowns (**State** → **District** → **City** and **Category** → **SubCategory**).

Phase 2: Authentication & Multi-step Onboarding

- **Task 2.1:** Implement standard User Registration & Login views.
- **Task 2.2:** Build the 3-step Expert Registration Wizard:
 - *Step 1:* Account basics (Email, Password, Phone).
 - *Step 2:* Professional credentials (Category, Experience, Location, Bio, Hourly Rate).
 - *Step 3:* Preview summary & final commit inside an atomic database transaction.
- **Task 2.3:** Set up role-based redirect access control (Users → User Dashboard, Experts → Expert Dashboard).

Phase 3: Expert Scheduling & Dynamic Search

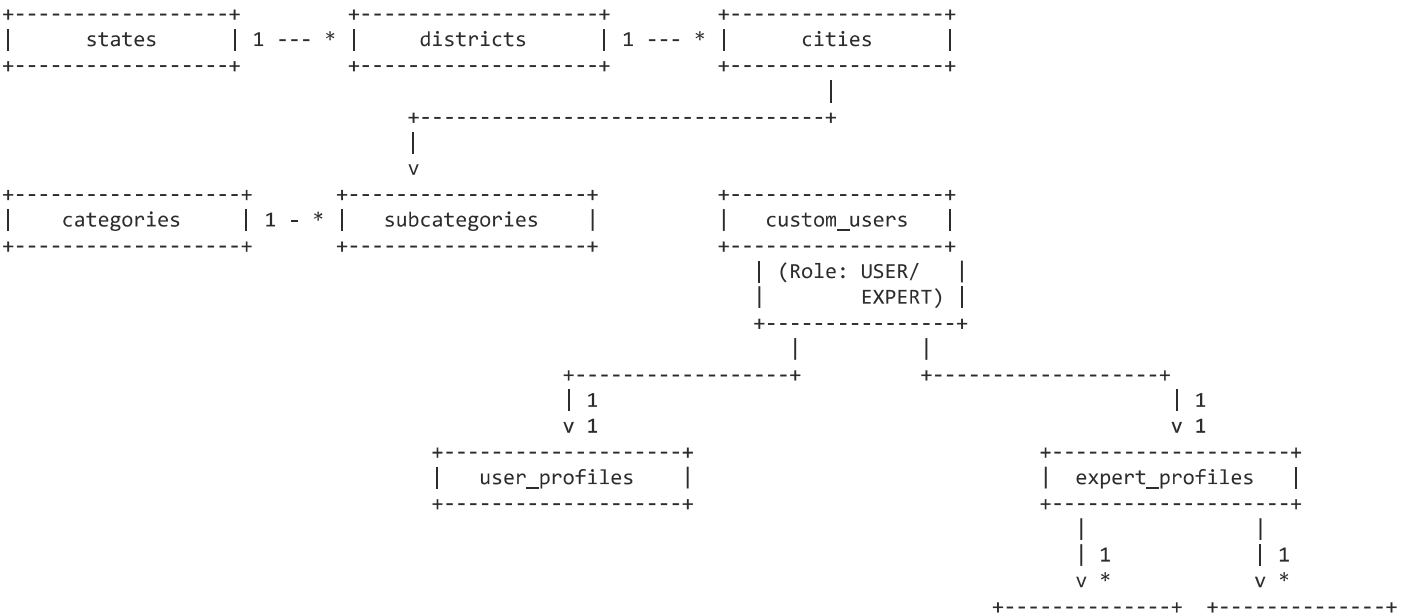
- **Task 3.1:** Build the Expert Schedule Manager where experts can open date and time slots with pricing.
- **Task 3.2:** Build the User Search Engine with filters for Category, Subcategory, Location, Maximum Hourly Rate, and Slot Availability.
- **Task 3.3:** Design the Expert Profile detail page where users can view credentials and request a session slot.

Phase 4: Booking Engine, Notifications & 24-Hour Expiry

- **Task 4.1:** Build the Booking Request pipeline (**User Requests** → **Notification created for Expert**).
- **Task 4.2:** Implement Expert Decision controls: **Accept**, **Reject**, or **Postpone**.
- **Task 4.3:** Build slot-locking mechanisms to prevent double booking.
- **Task 4.4:** Write a custom Django management command or task runner to check for unpaid bookings exceeding 24 hours and automatically mark them as EXPIRED.

2. MySQL Database Architecture & Relationships

ER Diagram & Relational Mapping



Key Relationships Explained

1. Location Hierarchy: states 1:N districts 1:N cities.
2. Category Hierarchy: categories 1:N subcategories.
3. User Profiles: custom_users has a 1 : 1 relation to user_profiles or expert_profiles depending on their role.
4. Availability & Bookings: An expert_profile has many (1:N) availabilities and bookings. A booking links a specific user, an expert, and an availability slot.

3. SQL Table Creation Scripts

You can execute these SQL queries directly in MySQL workbench or let Django generate them via python manage.py makemigrations and python manage.py migrate.

CREATE DATABASE IF NOT EXISTS advise_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE advise_db;

```
-- 1. Location Tables
CREATE TABLE states (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE districts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  state_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  FOREIGN KEY (state_id) REFERENCES states(id) ON DELETE CASCADE
);

CREATE TABLE cities (
  id INT AUTO_INCREMENT PRIMARY KEY,
  district_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  FOREIGN KEY (district_id) REFERENCES districts(id) ON DELETE CASCADE
);

-- 2. Category Tables
CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE subcategories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
);

-- 3. Authentication & Custom User Table
CREATE TABLE custom_users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(150) NOT NULL UNIQUE,
  email VARCHAR(254) NOT NULL UNIQUE,
  password VARCHAR(128) NOT NULL,
  phone_number VARCHAR(15) NOT NULL UNIQUE,
  role ENUM('USER', 'EXPERT') NOT NULL DEFAULT 'USER',
  is_staff TINYINT(1) DEFAULT 0,
  is_active TINYINT(1) DEFAULT 1,
  date_joined DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 4. User Profile
CREATE TABLE user_profiles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT UNIQUE NOT NULL,
  state_id INT,
  district_id INT,
  city_id INT,
  FOREIGN KEY (user_id) REFERENCES custom_users(id) ON DELETE CASCADE,
  FOREIGN KEY (state_id) REFERENCES states(id) ON DELETE SET NULL,
  FOREIGN KEY (district_id) REFERENCES districts(id) ON DELETE SET NULL,
  FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE SET NULL
);

-- 5. Expert Profile
CREATE TABLE expert_profiles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT UNIQUE NOT NULL,
  category_id INT,
  subcategory_id INT,
  qualification VARCHAR(255) NOT NULL,
  experience_years INT UNSIGNED NOT NULL,
  state_id INT,
  district_id INT,
  city_id INT,
  hourly_rate DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  bio TEXT,
```

```
FOREIGN KEY (user_id) REFERENCES custom_users(id) ON DELETE CASCADE,
FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL,
FOREIGN KEY (subcategory_id) REFERENCES subcategories(id) ON DELETE SET NULL,
FOREIGN KEY (state_id) REFERENCES states(id) ON DELETE SET NULL,
FOREIGN KEY (district_id) REFERENCES districts(id) ON DELETE SET NULL,
FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE SET NULL
);

-- 6. Expert Availability Schedule
CREATE TABLE expert_availabilities (
    id INT AUTO_INCREMENT PRIMARY KEY,
    expert_id INT NOT NULL,
    date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    is_booked TINYINT(1) DEFAULT 0,
    FOREIGN KEY (expert_id) REFERENCES expert_profiles(id) ON DELETE CASCADE
);

-- 7. Session Bookings
CREATE TABLE session_bookings (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    expert_id INT NOT NULL,
    slot_id INT NOT NULL,
    status ENUM('REQUESTED', 'ACCEPTED', 'POSTPONED', 'CONFIRMED', 'REJECTED', 'EXPIRED') DEFAULT 'REQUESTED',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    payment_deadline DATETIME NULL,
    FOREIGN KEY (user_id) REFERENCES custom_users(id) ON DELETE CASCADE,
    FOREIGN KEY (expert_id) REFERENCES expert_profiles(id) ON DELETE CASCADE,
    FOREIGN KEY (slot_id) REFERENCES expert_availabilities(id) ON DELETE CASCADE
);

-- 8. Notifications System
CREATE TABLE notifications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    message TEXT NOT NULL,
    is_read TINYINT(1) DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES custom_users(id) ON DELETE CASCADE
);
```

4. Complete Project File Structure

Here is the exact folder structure for your team to follow:

```
adwise_project/
├── manage.py                # Django CLI tool
├── requirements.txt         # Dependencies (Django, mysqlclient, pillow, etc.)
├── .env                     # DB credentials and secret keys
├── advise_core/             # Project Configuration Directory
│   ├── __init__.py
│   ├── settings.py         # MySQL DB settings, Installed Apps, Auth settings
│   ├── urls.py             # Main project URL routing
│   ├── wsgi.py
│   └── asgi.py
├── apps/                    # Modular Apps Folder
│   ├── __init__.py
│   ├── accounts/           # App 1: Auth & User Profiles
│   │   ├── migrations/
│   │   ├── __init__.py
│   │   ├── admin.py
│   │   ├── apps.py
│   │   ├── models.py       # CustomUser, UserProfile, ExpertProfile
│   │   ├── forms.py        # Multi-step registration forms
│   │   ├── urls.py
│   │   └── views.py        # Login, User Signup, Expert Wizard (Steps 1-3)
│   ├── locations/          # App 2: States, Districts, Cities & Dropdown APIs
│   │   ├── migrations/
│   │   ├── __init__.py
│   │   ├── models.py       # State, District, City models
│   │   ├── urls.py
│   │   └── views.py        # AJAX views returning JSON for dependent dropdowns
│   ├── categories/         # App 3: Categories & Subcategories
│   │   ├── migrations/
│   │   ├── __init__.py
│   │   ├── models.py       # Category, SubCategory
│   │   ├── urls.py
│   │   └── views.py        # Dynamic Subcategory dropdown API
│   ├── dashboard/          # App 4: Search Engine & Dashboards
│   │   ├── migrations/
│   │   ├── __init__.py
│   │   ├── urls.py
│   │   └── views.py        # Search views, Expert Filter Logic, Landing Page
│   └── bookings/           # App 5: Scheduling, Bookings & Expiry Logic
│       └── migrations/
```

